

TanStack Query

TanStack Query (formerly known as React Query) is often described as the missing data-fetching library for web applications, but in more technical terms, it makes **fetching, caching, synchronizing and updating server state** in your web applications a breeze.

This code snippet very briefly illustrates the 3 core concepts of React Query:

- Queries
- Mutations
- Query Invalidation

Query

- Query is a declarative dependency on an asynchronous source of data that is tied to a **unique key**. A query can be used with any Promise based method (**including GET and POST methods**) to fetch data from a server.

If your method modifies data on the server, we recommend using **Mutations** instead.

```
const { isPending, isError, data, error } = useQuery({  
  queryKey: ['todos'],  
  queryFn: fetchTodoList,  
})
```

The object contains a **few very important states** you'll need to be aware of to be productive. A query can only be in one of the following states at any given moment:

- **isPending or status === 'pending'** - The query has no data yet
- **isError or status === 'error'** - The query encountered an error
- **isSuccess or status === 'success'** - The query was successful and data is available

Beyond those primary states, more information is available depending on state of the query:

- **error** - If the query is in an **isError** state, the error is available via the error property.
- **data** - If the query is in an **isSuccess** state, the data is available via the data property.
- **isFetching** - In any state, if the query is fetching at any time (including background refetching) **isFetching** will be true.

Mutations

- mutations are typically used to **create/update/delete data or perform server side-effects**. For this purpose, TanStack Query exports a **useMutation** hook.

```
const mutation = useMutation({
  mutationFn: (newTodo) => {
    return axios.post('/todos', newTodo)
  },
})
```

A mutation can only be in one of the following states at any given moment:

- **isIdle or status === 'idle'** - The mutation is currently idle or in a fresh/reset state
- **isPending or status === 'pending'** - The mutation is currently running
- **isError or status === 'error'** - The mutation encountered an error
- **isSuccess or status === 'success'** - The mutation was successful and mutation data is available

Beyond those primary states, more information is available depending state of the mutation:

- **error** - If the mutation is in an error state, the error is available via the error property.
- **data** - If the mutation is in a success state, the data is available via the data property.

Mutation Side Effects

useMutation comes with some helper options that allow quick and easy side-effects at any stage during the mutation lifecycle.

```
useMutation({
  mutationFn: addTodo,
  onMutate: (variables) => { return { id: 1 } },
  onError: (error, variables, context) => { // An error happened!
    console.log(`rolling back optimistic update with id ${context.id}`)
  },
  onSuccess: (data, variables, context) => {},
  onSettled: (data, error, variables, context) => {},
})
```

Promises

- Use **mutateAsync** instead of **mutate** to get a promise which will resolve on success or throw on an error. This can for example be used to compose side effects.

```

const mutation = useMutation({ mutationFn: addTodo })
try {
  const todo = await mutation.mutateAsync(todo)
  console.log(todo)
} catch (error) {
  console.error(error)
} finally {
  console.log('done')
}

```

Role of QueryKey

- Think of queryKey as a **unique ID for the data** in the cache.
- **Why it's important:**
 - If two queries share the same queryKey, they **share the same cached data**
 - Changing the **queryKey** forces TanStack Query to treat it as a different dataset.
 - The queryKey can be **simple** (['todos']) or **complex with params** (['product', productId]).

```

useQuery({
  queryKey: ['todos', 'userA'],
  queryFn: fetchTodos,
});

useQuery({
  queryKey: ['todos', 'userB'],
  queryFn: fetchTodos,
});

```

These are treated as two separate queries, because the keys differ. Even though the function is the same, they fetch/cache independently.

Network Mode

- The networkMode option lets you **control the behavior of a query or mutation** based on the network availability (online/offline status).

Mode	Description
online	Default behavior. Runs only when the app is online. If a query runs because you are online, but you go offline

	while the fetch is still happening, TanStack Query will also pause the retry mechanism. If a query runs because you are online, but you go offline while the fetch is still happening, TanStack Query will also pause the retry mechanism.
always	Runs regardless of network status (even offline). Queries will never be paused because you have no network connection refetchOnReconnect defaults to false in this mode
offlineFirst	Runs immediately with cached data if offline, and syncs later when online. TanStack Query will run the queryFn once, but then pause retries. First fetch might succeed because it comes from an offline storage/cache. However, if there is a cache miss, the network request will go out and fail, in which case this mode behaves like an online query - pausing retries.

Disabling/Pausing Queries

- If you want to disable a query from automatically running, you can **use the enabled = false option**. The enabled option also accepts a callback that returns a boolean.

```
const { isLoading, isError, data, error, refetch, isFetching } = useQuery({
  queryKey: ['todos'],
  queryFn: fetchTodoList,
  enabled: !!filters, //True or False
})
```

Query Retries

- When a useQuery query fails (the query function throws an error), TanStack Query will **automatically retry the query** if that query's request has **not reached the max number** of consecutive retries (**defaults to 3**) or a function is provided to determine if a retry is allowed.
 - Setting **retry = false** will disable retries.
 - Setting **retry = 6** will retry failing **requests 6 times before showing the final error** thrown by the function.
 - Setting **retry = true** will infinitely retry failing requests.
 - Setting **retry = (failureCount, error) => ...** allows for custom logic based on why the request failed.

- TanStack Query do **not happen immediately after a request fails**. As is standard, a **back-off delay is gradually applied** to each retry attempt. The **default retryDelay** is set to **double (starting at 1000ms)** with each attempt, **but not exceed 30 secs**.

```
const result = useQuery({
  queryKey: ['todos', 1],
  queryFn: fetchTodoListPage,
  retry: 10,
  retryDelay: (attemptIndex) => Math.min(1000 * 2 ** attemptIndex, 30000),
})
```

Paginated Queries

- The data from the last **successful fetch is available** while **new data is being requested**, even though the query key has changed. When the **new data arrives**, the previous data is **seamlessly swapped to show the new data**.

```
const { error, data, isFetching, isPlaceholderData } =
  useQuery({
    queryKey: ['projects', page],
    queryFn: () => fetchProjects(page),
    placeholderData: keepPreviousData,
  })
```

Paginated Queries

- This is easy to fix, by always using the hook `useSuspenseQueries` when you have multiple suspenseful queries in a component.

```
const [usersQuery, teamsQuery, projectsQuery] = useSuspenseQueries({
  queries: [
    { queryKey: ['users'], queryFn: fetchUsers },
    { queryKey: ['teams'], queryFn: fetchTeams },
    { queryKey: ['projects'], queryFn: fetchProjects },
  ],
})
```

API References

1. **useQuery**
 - Fetches and caches server data.
2. **useMutation**
 - Create, Update, Delete Data. Handles POST, PUT, DELETE operations.
3. **useQueryClient**
 - Control the Cache. Gives you direct access to the Query Client (cache control).

Method	Description
<code>invalidateQueries</code>	Mark data as stale, triggers refetch
<code>setQueryData</code>	Manually update cached data
<code>getQueryData</code>	Read cached data
<code>removeQueries</code>	Remove queries from cache
<code>prefetchQuery</code>	Prefetch and cache data without showing it

4. **QueryClient**
 - Holds all cached data, configurations, and manages global state.
5. **QueryClientProvider**
 - Wrap your app to enable TanStack Query.
6. **useQueries**
 - Run **multiple independent queries** simultaneously and manage them together.

```
const queries = useQueries({
  queries: ids.map((id) => ({
    queryKey: ['post', id],
    queryFn: () => fetchPost(id),
  })),
  combine: (results) => {
    return {
      data: results.map((result) => result.data),
      pending: results.some((result) => result.isPending),
    }
  },
});
```

7. **Hydration**
 - Sharing pre-fetched data from the server to the client, so the client doesn't need to refetch that data on page load.